



CodeMentor: Assistant de code

Nom de l'apprenti/-e : Thibaud Gamez

N° de l'apprenti/-e : 151446

Description générale du projet d'examen :

CodeMentor est une application web d'analyse et d'assistance au développement. Après authentification, l'utilisateur peut créer un projet lié à un dépôt GitHub. L'application analyse automatiquement le code, génère des explications, détecte des erreurs potentielles, propose des corrections et crée des visualisations graphiques avec Mermaid pour faciliter la compréhension du projet.

Domaine du TPI : (Cocher la case de couleur correspondante)

☒ Développement	☐ Réseau	☐ Systèmes
---	---------------------------------	-----------------------------------

Tracé d'activité de l'apprenti/-e durant la dernière année d'apprentissage :

- Un semestre en stage en entreprise
- Un semestre en travaux sur mandats (projets) en phase de professionnalisation à l'EMF

Coordonnées de l'apprenti/-e :

E-Mail : thibaud.gamez@gmail.com
Tél : 079 600 04 28
Prénom : Thibaud
Nom : Gamez
Adresse : École des métiers de Fribourg
Chemin du musée 2
1700 Fribourg

Le supérieur professionnel

Silvin Meylan

Lieu et Date

Fribourg, le 9 février 2026

Table des matières

1	DONNÉE DU PROBLÈME.....	3
2	FORMULATION DU MANDAT – ÉTAT DÉSIRÉ	3
2.1	DESCRIPTION ET OBJECTIFS DU PROJET	3
2.2	EXIGENCES FONCTIONNELLES	5
2.3	ANALYSE.....	6
2.4	CONCEPTION.....	6
2.5	RÉALISATION.....	6
3	INFRASTRUCTURE NÉCESSAIRE.....	6
4	CONNAISSANCES PRÉALABLES	6
5	TRAVAUX PRÉPARATOIRES.....	7
6	STANDARDS D'ENTREPRISE	7
7	DOCUMENTATION OBLIGATOIRE	8
8	CRITÈRES D'ÉVALUATION.....	10
9	DÉLAIS	10

1 Donnée du problème

La compréhension et l'analyse d'un projet logiciel existant représentent souvent un défi, surtout lorsque le code est complexe ou peu documenté. Les développeurs doivent passer de nombreuses heures à explorer les fichiers, comprendre les dépendances et identifier les erreurs potentielles.

Le manque de clarté dans la structure du code et la documentation entraîne une perte de temps et une augmentation des risques d'erreurs lors de la maintenance.

L'objectif du projet **CodeMentor** est de simplifier ce processus en proposant une solution intelligente et interactive. L'application analysera automatiquement le dépôt GitHub d'un projet, détectera les erreurs et proposera des corrections, tout en générant des explications textuelles et des diagrammes de structure pour aider à la compréhension du code.

2 Formulation du mandat – état désiré

2.1 Description et objectifs du projet

Application frontend

L'interface utilisateur doit permettre :

- La connexion sécurisée à l'application (authentification par identifiants, OAuth ou base interne).
- La création et la gestion de projets, chacun lié à un dépôt GitHub.
- L'affichage des résultats d'analyse : explications du code, erreurs détectées, suggestions de correction.
- La visualisation graphique de la structure du projet via des diagrammes générés avec Mermaid.js.
- Une interface ergonomique, claire et réactive.

Application backend

Le backend gèrera :

- Une API REST pour la gestion des utilisateurs, projets et résultats d'analyse.
- L'intégration avec l'API GitHub pour récupérer le contenu du dépôt.
- L'utilisation de l'API OpenAI (ou équivalente) pour générer les explications et suggestions.
- La mise en place d'un mécanisme de traitement asynchrone pour les tâches longues (analyse du code via l'IA).
 - Lorsqu'un utilisateur lance une analyse, le backend crée une tâche et la place dans une file d'attente interne.
 - Un processus en arrière-plan traite les tâches de manière séquentielle ou contrôlée, sans bloquer les requêtes HTTP de l'application.
 - L'état d'avancement de chaque tâche est stocké et accessible via l'API.
 - Le frontend interroge régulièrement l'API afin de suivre la progression de l'analyse et de récupérer le résultat final.
- La documentation du backend via Swagger.

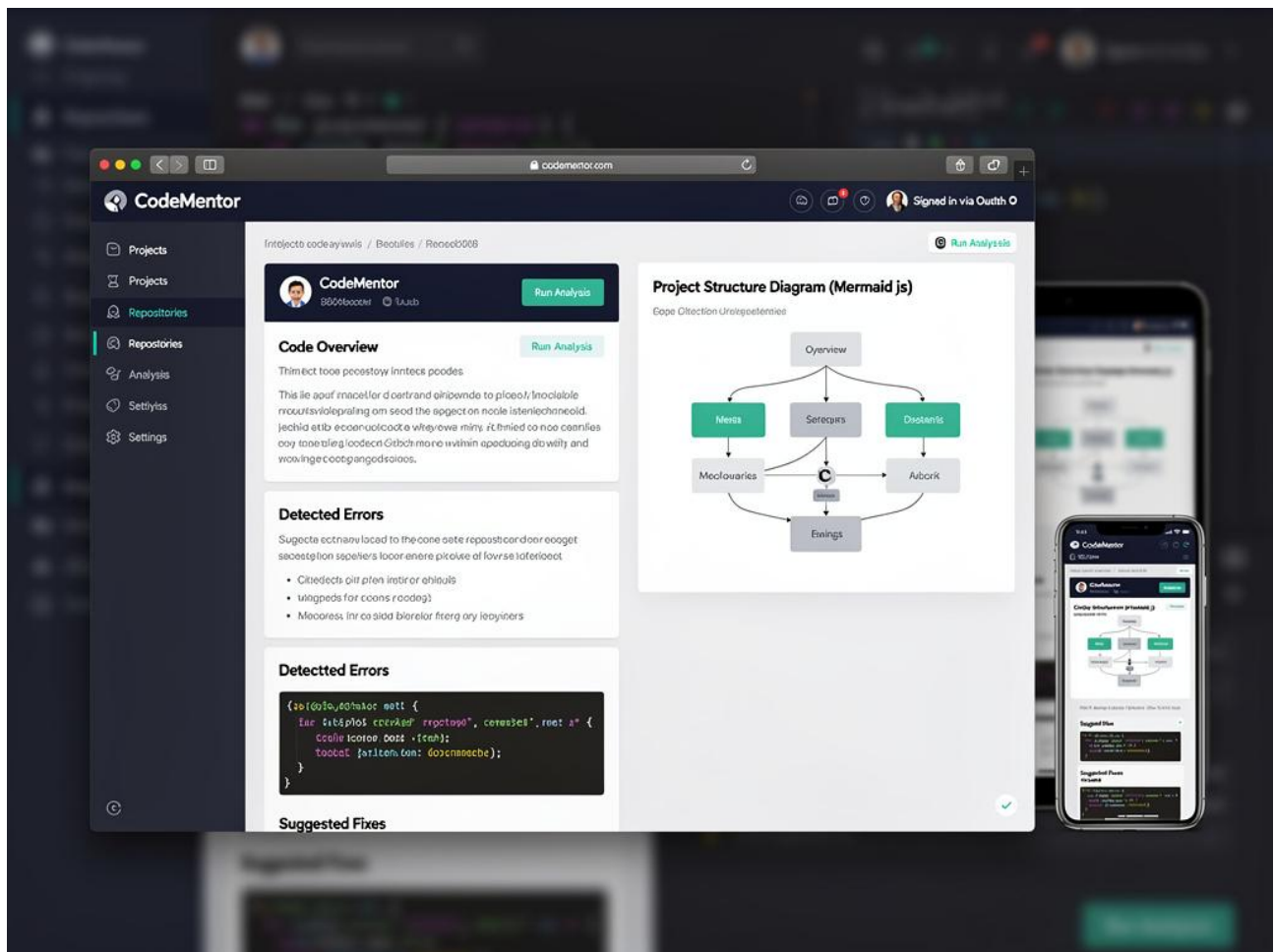


Figure 1 : Maquette exemple de l'application

2.2 Exigences fonctionnelles

Exigence fonctionnelle	Description	Priorité Haute Moyenne Basse	Temps estimé (jours)
Lecture cahier des charges	L'apprenti/-e doit comprendre l'énoncé, la portée et la complexité du projet avant toute autres actions. Dès lors une mise en place de tous les documents utiles sont à réaliser.	Haute	0.5 jour
Analyse des besoins	Réalisation des diagrammes UML d'analyse sur la base des maquettes mises en place avec le client avant le début du travail. Réalisation d'une documentation d'analyse.	Haute	0.5 jour
Planning	Réalisation d'une planification détaillée ainsi qu'un suivi, représentant les différentes tâches, de l'analyse, de la conception, de l'implémentation, des tests et de la documentation, à réaliser dans ce projet.	Haute	0.5 jour
Conception	Réalisation les diagrammes UML de conception permettant de représenter l'architecture des deux l'applications et de leur association. Réalisation du modèle ER de la base de données. Réalisation d'une documentation de conception	Haute	1 jour
Implémentation de l'application Backend	Implémentation de l'application Backend et de ses appels à la base de données.	Haute	3 jours
Implémentation de l'application FrontEnd	Implémentation de l'application Frontend et ses appels à l'application Backend	Haute	2.5 jours
Tests fonctionnels	Réalisation des tests fonctionnels des applications et des accès à la base de données en situation de production	Moyenne	0.5 jour
Documentation	Tout au long du projet, la documentation sera mise à jour régulièrement et finalisée après chaque phase de développement terminée. Le dernier jour permettra de finaliser les différentes documentations du projet avant de les rendre selon les consignes demandées.	Haute	1.5 jour

2.3 Analyse

Cette phase de travail permet d'analyser les besoins tels qu'ils ont été décrits dans la donnée du problème, en les représentant par des diagrammes ou schémas adéquats.

Durant la phase d'analyse, l'apprenti/-e accomplit les tâches suivantes :

- Analyser les tâches complètes du projet et les maquettes préparées auparavant.
- Créer le diagramme des cas d'utilisations global.
- Créer les différents diagrammes de séquences des systèmes pour les applications.
- Créer les différents diagrammes d'activités principaux pour les applications.
- Documenter l'analyse.

2.4 Conception

Durant la phase de conception, l'apprenti/-e accomplit les tâches suivantes :

- Créer le modèle relationnel de la base de données utile pour l'application Backend.
- Créer le diagramme des classes des applications.
- Créer les différents diagrammes d'interactions des méthodes principales des applications.
- Documenter la conception.

2.5 Réalisation

Durant la phase de réalisation, l'apprenti/-e accomplit les tâches suivantes :

- Réaliser l'implémentation de la base de données.
- Réaliser les différentes requêtes SQL utiles pour la base de données.
- Réaliser l'implémentation des services REST (application BackEnd) en Java ou PHP ou NodeJS à choix
- Développer l'interface homme-machine (application FrontEnd).
- Réaliser l'implémentation des applications et leur communication.
- Tester le fonctionnement complet des applications et leur relation.
- Documenter la réalisation

3 Infrastructure nécessaire

Pour la réalisation du projet, l'apprenti/-e disposera de l'infrastructure suivante :

- Ordinateur personnel Windows avec :
 - o Connection à Internet, navigateurs Internet de différents fabricants et MS-Office.
 - o MS-Windows 10 minimum.
 - o Logiciel de développement VSCode (ou autre) ainsi que des plug-ins nécessaires.
 - o Entreprise Architect pour d'analyse et la conception UML.
 - o Console MySQL WorkBench

4 Connaissances préalables

Les technologies des différents thèmes pour les applications de ce projet, auront été étudiées avant le début du projet. Pour la partie frontend, le Framework Javascript moderne et à tester avant le projet TPI et la génération de schéma mermaid également.

Pour la partie backend, il faut avoir testé NodeJS ou Springboot ou PHP avec des requêtes sur une DB. Il faut également avoir testé la librairie pour atteindre Github et celle pour OpenAI. Finalement il faut savoir faire une queue en mémoire pour une exécution asynchrone (pour tâches longue comme appel à l'API de OpenAI).

Pour le reste des compétences qui seront à utiliser dans ce projet le candidat a été formé dans les différents modules de sa formation à l'école.

5 Travaux préparatoires

- Tests technologiques :
 - JavaScript: framework VueJS
 - NodeJS ou PHP ou Java
 - avec accès à MySQL.
 - Accès API Github
 - API OpenAI.
 - Swagger.
 - Queue en mémoire (job asynchrone simple)

6 Standards d'entreprise

- UML comme langage de modélisation de logiciel.
- Modélisation relationnelle des bases de données.
- Architecture Pattern MVC ou parente.
- Documentation selon les normes apprises dans le modèle de l'école.
- Planification, ToDoList des tâches et journal de travail à utiliser selon les normes apprises dans la gestion de projet à l'école

7 Documentation obligatoire

A la fin du projet, l'apprenti/-e doit fournir les documents suivants :

Une planification

Cette planification doit être réalisée au début du projet avant toute autre action (selon modèle fourni). Elle décrit les étapes importantes du projet ainsi que la durée estimée correspondante. Elle doit être validée par le supérieur professionnel.

Un journal de travail

Ce document décrit les diverses étapes et activités liées au projet (selon modèle fourni).

Une documentation d'analyse

Ce document détermine les exigences et contraintes du projet et permet la justification des choix pour la réalisation du travail demandé. Ce document est composé de :

1. Synthèse de la définition du projet et des choix définitifs.
2. Explications de tous les diagrammes d'analyse réalisés.

Une documentation de réalisation

La documentation de réalisation a pour objectif de faciliter la maintenance et doit contenir les informations suivantes :

1. Conception :
 - a. Les diagrammes de classes des applications.
 - b. Les diagrammes d'interactions des tâches principales des applications
 - c. Le modèle ER de la base de données de l'application Backend
2. Implémentation :
 - a. Les codes sources des applications, commentés.
 - b. Le script de création et sauvegarde de la base de données, commenté.
 - c. Tests fonctionnels des applications et leur communication.
3. Remarques et la conclusion :
 - a. Problèmes rencontrés, limites des versions et améliorations possibles.
 - b. Commentaires personnels et une auto-évaluation.

Une documentation d'utilisation

Ce document a pour objectif de permettre à l'utilisateur de se former d'une manière autodidacte et doit contenir les informations suivantes :

1. Installation et configuration des applications.
2. Utilisation de chaque application

Un Web Summary

Ce document a pour objectif de présenter le projet de manière succincte.

La remise de la documentation

La documentation est remise selon les instructions du manuel ICT-FR, partie C, point 1.9.1 « Périmètre du rapport, Tips ».

Le manuel est disponible dans PkOrg sous : Documents ➔ Documents pour tous :

<http://manuel.ict-fr.ch>

8 Critères d'évaluation

<http://criteres.ict-fr.ch>

Compétences professionnelles globales (selon la grille d'appréciation)

Résultat et efficience

1. **Solution / Produit** (selon la grille d'appréciation)
2. **Critères spécifiques au projet**
 - 146 - Satisfaction utilisateur
 - 225 - Gestion des versions avec un logiciel d'administration
 - 235 - Projet avec UML
 - 164 - Codage : Traitement des erreurs
 - 165 - Implémentation de solutions (Programmation)
 - 166 - Style de codage ; Lisibilité du code
 - 250 - Séparation des couches (Application)
3. **Documentation** (selon la grille d'appréciation)
4. **Journal de travail** (selon la grille d'appréciation)

Présentation et entretien professionnel (selon la grille d'appréciation)

9 Délais

Le projet y compris la remise du projet aux experts se terminera le 8 juin 2026 à 12 heures. Le dernier délai pour la présentation du projet est fixé au 17 juin 2026 à 12:00.

La documentation est déposée sur PkOrg dans les temps.